

Write-up: Support Ticket Triage Agent

Natcha Soranathavornkul

Architecture decisions and why

I built a small tool-calling agent: the model chooses which tools to call, my code runs them, and the results go back to the model until it submits a decision. No framework, because the loop is only about 100 lines and I wanted the logic easy to read. A single prompt wouldn't be a real agent, and a fixed pipeline would cost more calls for no better decisions at this scale.

There are four tools: customer history, knowledge-base search, billing and system status. Anything computable is computed in the tools, not guessed by the model. The billing tool totals charges and flags duplicates, and the status tool flags when the public status page disagrees with internal health. That is how the agent catches the outage hiding behind "all systems operational" in the Thai ticket.

The final answer is also a tool call, `submit_triage_decision`, validated against a Pydantic schema. If it's invalid, the model sees the error and retries. A round limit caps cost, and the last round forces a submission.

I use the Responses API. My first version used Chat Completions, which OpenAI rejected for function tools combined with reasoning on my chosen model. The model name is an environment variable, so it works with the reviewer's key.

The rules that matter most don't rely on the model. Seven plain-Python guardrails review every decision. Critical tickets, dispute or legal wording, and urgent Enterprise tickets go to a human; angry customers are never auto-answered; low confidence goes to a human; an auto-reply must cite a real knowledge-base article; and team and action must agree. A guardrail can only make a decision more cautious, and a test checks that.

The OpenAI client is injected, so the tests run offline against a scripted fake. Tool back-ends are JSON files and KB search is weighted keywords: mocks I can replace one file at a time.

What could go wrong

- **A wrong call on an urgent ticket** is the worst case, so every guardrail pushes toward a human, and the prompt rates urgency by business impact, not tone. In ticket 1 the customer is shouting, but it's high, not critical.
- **Invented answers or promised refunds.** The prompt forbids both, and auto-replies must cite a real article. The gap: that proves the article exists, not that it supports the reply.
- **Prompt injection.** Ticket text is fenced as data, tools are read-only and output is schema-checked, so a successful injection can't do much.
- **Tool or API failures** return to the model as errors it can recover from. Once it queried the status tool with "Asia", got the valid regions back and retried with "TH". Requests have timeouts and retries, and one failed ticket doesn't stop the batch.
- **Run-to-run variance.** Ticket 3's sentiment flipped between neutral and positive across runs, while urgency and action stayed stable.
- **Known gaps:** replies say a feature request was "recorded" but nothing records it, and ticket text goes to a third-party API without PII redaction.

How I'd evaluate it in production

Before launch I'd build a labelled set of about 200 tickets, including other languages, vague messages and injection attempts, and re-run it on every prompt or model change. The number I'd watch most is under-escalation: tickets that needed a human but got an automatic reply. I'd also check replies for language, unsupported promises and whether the cited article backs them up (LLM judge plus human spot checks), and repeat each ticket a few times to measure consistency.

I'd roll out in shadow mode first, with the agent suggesting and a person deciding, then allow auto-replies only for low-risk categories with sampled audits. Live, I'd track human override rate, guardrail trigger rate (a rise means drift), escalation rate, reopen rate and CSAT on auto-answered tickets, time to first response, and cost and tool rounds per ticket.